Implementación de modelo de lenguaje tipo GPT en Mojo

Tomas Vicente
Institut TIC de barcelona
tomas240605@gmail.com

Abstract

Este proyecto presenta la implementación de un modelo de lenguaje autorregresivo ligero basado en arquitecturas transformer de tipo GPT. Siendo el objetivo principal comprender en profundidad el funcionamiento interno de los modelos de lenguaje modernos mediante la construcción de una versión simplificada desde cero, en lugar de centrarse en el rendimiento a gran escala. El modelo ha sido desarrollado en Mojo sin el uso de frameworks de deep learning como PyTorch, e incluye la implementación manual de componentes fundamentales como embeddings de tokens, mecanismos de atención, capas feed-forward y el proceso de predicción autoregresiva de tokens. La arquitectura se basa en las implementaciones de MicroGPT [1] y NanoGPT [2] hechas por Andrej Karpathy.

1 Introducción

Los Large Language Models (LLMs) se han convertido en uno de los avances más relevantes en el Procesamiento del Lenguaje Natural (NLP). Las arquitecturas basadas en transformers, especialmente los modelos tipo GPT, son capaces de aprender relaciones entre tokens mediante el contexto y llegan a generar texto de manera coherente mediante predicción autoregresiva.

Este proyecto explora la implementación de un modelo de lenguaje autorregresivo simple basado en las creaciones de Andrej Karpathy como MicroGPT [1] y NanoGPT [2]. El objetivo principal no fue competir con modelos comerciales de gran escala, sino entender en profundidad los componentes fundamentales que conforman estos modelos.

La implementación fue desarrollada completamente en Mojo, sin utilizar frameworks de aprendizaje como PyTorch. Para ello, se construyeron manualmente los elementos esenciales para el modelo, incluyendo un tokenizador, embeddings, sistemas de atención, lineares y optimización. De esta forma, el proyecto busca un entendimiento a detalle de los modelos transformer y el proceso de entrenamiento de sistemas generativos de lenguaje.

2 Arquitectura

El modelo implementado sigue una arquitectura simplificada de tipo transformer autorregresivo inspirada en los modelos GPT. Su diseño se basa en la composición de bloques secuenciales que permiten transformar una cadena de tokens de entrada en una distribución de probabilidad sobre cuál es el siguiente token.

En primer lugar, la entrada del modelo consiste en una secuencia de tokens que son transformados en representaciones vectoriales mediante una capa de embeddings. Esta representación permite que el modelo trabaje en un espacio numérico denso donde puede aprender relaciones semánticas y sintácticas entre los tokens.

Posteriormente, las representaciones con embeddings son procesadas con una serie de bloques transformer. Cada bloque está compuesto por dos componentes principales: un mecanismo de atención y una red neuronal feed-forward. El mecanismo de atención permite al modelo calcular la importancia relativa de cada token respecto al resto de la cadena, capturando dependencia a largo plazo dentro del contexto.

El funcionamiento de la atención se basa en la proyección de las entradas en tres espacios diferentes: queries, keys y values.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

A partir de estas proyecciones, se calculan puntuaciones de atención que determinan qué información debe ser destacada en cada posición de la cadena. Este proceso permite al modelo adaptar dinámicamente su representación interna en función del contexto.

Cada bloque transformer incluye además una red feed-forward completamente conectada aplicada de forma independiente a cada token, lo que introduce no linealidad adicional al modelo y mejora su capacidad de representación.

En cuanto a la configuración concreta, el modelo implementado consta de 2 capas transformer, cada una con 2 cabezas de atención y una dimensión oculta de 32. Esta configuración se ha elegido deliberadamente por motivos educativos y de eficiencia computacional, permitiendo observar el comportamiento y sus resultados sin requerir grandes equipos.

Hiperparámetro	**Valor configurado**
Capas Transformer	2
Cabezas de atención	2
Dimensión del embedding	32
Tamaño del contexto	64 tokens
Tamaño del vocabulario	256 Basado en ASCII

Finalmente, la salida del modelo pasa por una capa lineal seguida de una función softmax, que produce una distribución de probabilidad sobre el vocabulario. Esta distribución se utiliza para realizar la predicción del siguiente token de forma autorregresiva.

El entrenamiento del modelo se realiza mediante la minimización de la función de pérdida cross entropy loss [3], optimizada con el algoritmo Adam [4], lo que permite ajustar los parámetros del modelo de forma estable durante el proceso de aprendizaje.

3 Entrenamiento

El entrenamiento del modelo se realizó utilizando un enfoque autorregresivo de predicción del siguiente token, siguiendo el mismo principio fundamental empleado en arquitecturas GPT modernas. Durante este proceso, el modelo recibe una cadena de tokens y aprende a predecir cuál será el siguiente en función del contexto previo.

Para ello, el texto utilizado durante el entrenamiento fue tokenizado anteriormente transformando en secuencias numéricas compatibles con la arquitectura. Cada token de entrada es procesado por el modelo, generando una distribución de probabilidad sobre el vocabulario completo. Posteriormente, dicha predicción se compara con el token esperando mediante una función de pérdida, cross entropy loss.

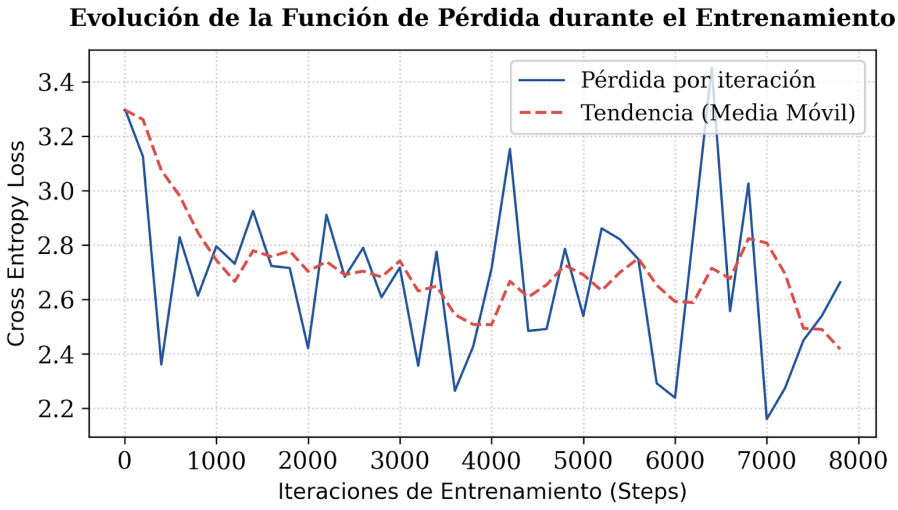
La función de pérdida indica que el modelo está aprendiendo patrones presentes en los datos.

La optimización de los parámetros se llevó a cabo mediante el algoritmo Adam, ampliamente utilizado en tareas de deep learning debido a su estabilidad y eficiencia durante el entrenamiento. Este optimizador ajusta dinámicamente la tasa de aprendizaje en cada parámetro utilizando estimaciones adaptadas al gradiente.

Durante las pruebas realizadas, el modelo mostró una convergencia estable y una disminución progresiva de la función de pérdida, indicando que la arquitectura era capaz de aprender relaciones internas dentro de los datos de entrenamiento. Tras varias iteraciones, el modelo comenzó a estructurar secuencias coherentes y estructuras similares a las presentes en el conjunto de datos original.

4 Resultados

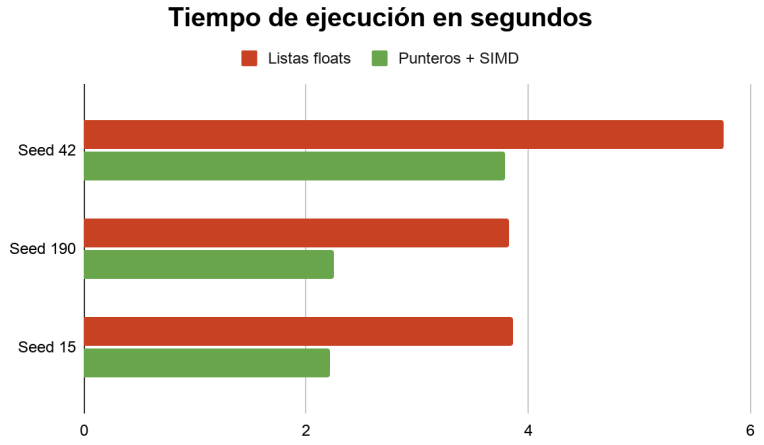
Para ver cómo aprende el modelo, registramos el valor de la pérdida (loss) a lo largo de 7,800 iteraciones y tomamos muestras de los nombres que el MLP iba generando en diferentes momentos. Al principio, en el step 0, la loss arranca en su punto más alto (3.2956). En este punto, los pesos de la red son totalmente aleatorios; aunque la terminal muestra palabras del dataset al muestrear, la generación no tiene ninguna estructura fonética real.Sin embargo, el aprendizaje es bastante rápido.



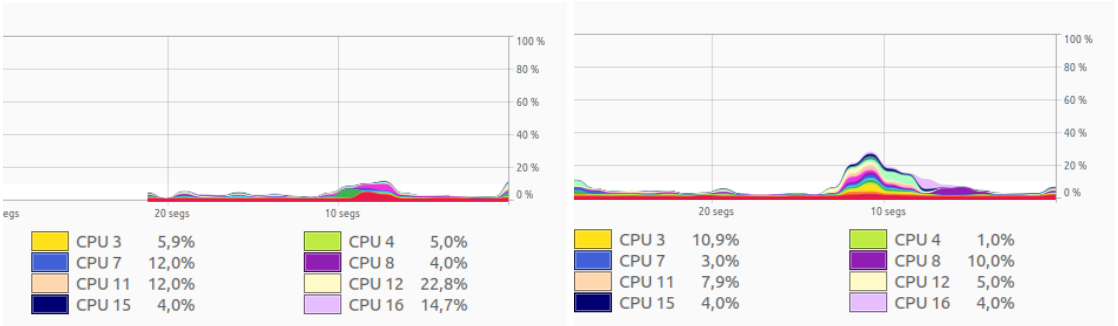
La loss cae de forma brusca en las primeras iteraciones y ya en el step 400 toca un mínimo de 2.3613. A partir de ahí, la curva empieza a oscilar de arriba a abajo en una especie de meseta (moviéndose entre 2.2 y 2.9). Lo importante es comprobar la tendencia general del modelo. La media móvil demuestra que la red sigue refinando sus parámetros y buscando mínimos estables. De hecho, el mejor resultado absoluto ocurre en el step 7000, donde la loss baja hasta 2.1603. Si analizamos los nombres generados a nivel cualitativo, el cambio es radical. Mientras que al principio todo dependía del token inicial, en la segunda mitad del entrenamiento el modelo empieza a encadenar sílabas naturales y a respetar las terminaciones típicas del corpus, llegando a generar nombres limpios y con perfecta sonoridad como marisol (step 3600) o raphael (step 7800).

Step	Loss	Nombre generado	Evaluación
0	**3.2956**	**Zacarias**	**Alta dependencia del token inicial**
400	**2.3613**	**Lillianah**	**Consigue estructuras silábicas compuestas**
3600	**2.2644**	**Marisol**	**Aprende combinaciones de sílabas comunes del dataset**
7000	**2.1603**	**Adalia**	**Pérdida mínima registrada**
7800	**2.6632**	**Raphael**	**Estabilización final del modelo**

El impacto de usar SIMD otro gran pilar de este proyecto era comprobar si el uso de instrucciones SIMD (Single Instruction Multiple Data) y la gestión manual de memoria en Mojo realmente marcaban una diferencia frente a la estructura clásica. Para asegurarnos de que los resultados eran sólidos y no dependían de un golpe de suerte, ejecutamos el código usando tres semillas aleatorias distintas (seed 42, seed 190 y seed 15).



En el caso de la seed 42, pasar de la versión inicial con listas de floats a la versión optimizada con punteros y SIMD, redujo el tiempo de ejecución de 5.76 a 3.79 segundos (un 34.2% más rápido). Esta ganancia se vuelve todavía más clara en las otras dos semillas (seed 190 y seed 15), donde los tiempos de la versión inicial bajan de los 3.8 segundos hasta quedarse en apenas 2.2 segundos, logrando aceleraciones que rozan el 42%. En promedio, la optimización reduce el tiempo total de ejecución en un 38.61%. Mirando el monitor del sistema durante las pruebas, la explicación de esta mejora es puramente física.



En la versión inicial, el procesador trabaja de manera asíncrona y tiende a saturar hilos sueltos mientras los demás se quedan en niveles mínimos (por ejemplo, el hilo CPU 12 asume casi todo el esfuerzo con un 22.8% de uso). En cambio, al activar SIMD, la carga cambia por completo: el bucle que calcula las multiplicaciones de matrices se vectoriza en el silicio, lo que obliga a los núcleos de la CPU a trabajar en paralelo y de forma

equilibrada (repartiendo el esfuerzo en torno al 10% en hilos como CPU 3, CPU 8 y CPU 11 simultáneamente). Esto demuestra que Mojo permite exprimir la potencia del procesador de forma nativa sin necesidad de añadir frameworks externos pesados.

5 Conclusión

En este trabajo se ha desarrollado una implementación completa de un modelo de lenguaje autorregresivo tipo Transformer en Mojo puro, tomando como base el diseño de MicroGPT [1]. Se ha logrado unificar en un único fichero mojo todos los componentes esenciales del sistema: el tokenizador por caracteres, el sistema de atención, el ciclo de entrenamiento y el motor de inferencia, eliminando por completo la dependencia de frameworks externos de deep learning.

Las pruebas de rendimiento demuestran que el uso de primitivas de bajo nivel y la vectorización mediante instrucciones SIMD son fundamentales para exprimir la capacidad del procesador. Esta optimización distribuye la carga de trabajo de forma paralela y equilibrada entre los núcleos de la CPU, reduciendo los tiempos de ejecución en casi un 40% y agilizando drásticamente la velocidad en la síntesis de nombres propios.

De cara al futuro, y a medida que el ecosistema de Mojo continúe madurando, la línea de desarrollo lógica consiste en migrar la gestión manual de memoria basada en punteros hacia una implementación nativa fundamentada en tensores dinámicos. Esto dotará al modelo de una flexibilidad estructural muy superior, facilitando el escalado de la arquitectura para trabajar con conjuntos de datos más grandes y complejos.

6 Referencias

[1] A. Karpathy.
microGPT.
GitHub repository,
https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95, 2023.

[2] A. Karpathy.
NanoGPT.
GitHub repository,
https://github.com/karpathy/nanogpt, 2023

[3] A. Mao, M. Mohri, and Y. Zhong.
Cross-Entropy Loss Functions: Theoretical Analysis and Applications.
arXiv preprint arXiv:2304.07288, 2023.

[4] D. P. Kingma and J. Ba.
Adam: A Method for Stochastic Optimization.
arXiv preprint arXiv:1412.6980, 2014.